

G1 Locomotion 인수인계

Unitree RL Lab을 활용한 Locomotion policy 학습

1. 학습 코드 구성

대부분 isaacsim의 코드를 가져와 사용; 기본적인 구조 유사

- **velocity_env_cfg:** source/unitree_rl_lab/unitree_rl_lab/tasks/locomotion/robots/g1/29dof/
terrain, reward, obs 등 sim environment 구성
isaacsim의 source/isaacsim/isaacsim/envs/mdp에서 각 cfg의 원형 확인 가능.
unitree rl lab 역시 source/unitree_rl_lab/unitree_rl_lab/tasks/locomotion/mdp 에서 기존 isaacsim에
서 추가한 내용 확인 가능
 - EASY_ROUGH_CFG: isaacsim의 rough_terrain_cfg를 가져와 height 등의 parameter를 수정한 것
 - RobotSceneCfg: 어느 terrain을 사용할지 2. Rewards정할 수 있음
 - EventCfg: reset_base에서 reset position을 설정할 수 있음 → initial pose randomization 가능
(pose_range의 z, roll, pitch 추가함). 그외 다양한 event 설정
 - CommandsCfg: range에서 초기 velocity command, limit_ranges에서 max/min 값을 설정. 현재는
x방향 positive velocity command와 ang_vel만 설정했지만 후진 및 y방향 command도 추가 가능.
 - ActionsCfg: scale = 0.25로 설정되어 있음. Target action = policy output * scale + offset
 - ObservationsCfg: Actor(Policy)와 Critic의 observation 설정. history length 설정.
 - RewardsCfg: 사용할 reward와 weight 설정
 - TerminationsCfg: termination 할 조건 설정
 - time_out: 정해진 episode length가 끝나는 경우
 - base_contact: torso에 contact이 감지된 경우
 - base_height: torso가 기준 height보다 낮게 위치한 경우 → flat terrain에서만 유효
 - bad_orientation: projected gravity의 각도가 지정한 값보다 큰 경우
 - CurriculumCfg: terrain과 velocity command level 조절을 위한 설정
 - RobotEnvCfg: Sim setting. episode_length_s로 시간 조절.
 - RobotPlayEnvCfg: RobotEnvCfg를 상속받아 play 용으로 사용. play 시 default num_envs 및
spawn할 terrain 설정
- **rsl_rl_ppo_cfg:** source/unitree_rl_lab/unitree_rl_lab/tasks/locomotion/agents/
policy network 구성
 - max_iterations: default iteration 설정
 - actor/critic layer dimension 설정: history length와 height scanner 값으로 인해 기존보다 키움.

- learning rate, mini batch num, clip_actions, logging 설정
- **train.py:** scripts/rsl_rl/
line 196-197 dump_pickle 주석처리함 (사용 X)
- **play.py:** scripts/rsl_rl/
play and log
play 실행 시 wandb에 unitree_rl_lab_play라는 project를 생성하고 기록.
현재 logging 항목은
 - actions(policy output)
 - processed_actions: scale과 offset 처리 한 후의 실제 target position
 - real_action: 실제 로봇의 joint position
 - joint_vel: 실제 로봇의 joint velocity
 - torque: 실제 로봇의 applied torque
 그밖에 logging 할 수 있는 항목은 isaacclab의
source/isaacclab/isaacclab/assets/articulation/articulation_data.py에서 확인 가능
- **unitree.py:** source/unitree_rl_lab/unitree_rl_lab/assets/robots/
line 20: unitree model dir 설정
line 397: UNITREE_G1_29DOF_CFG에서 joint의 initial position, stiffness, damping 등을 설정 →
train.py 실행 시 logs/rsl_rl/unitree~~~/2025-xx-xx_xx-xx-xx/params/deploy.yaml에 기록→
실제 로봇에 사용될 deploy/robots/g1_29dof/config/policy/velocity/v0/params/deploy.yaml을 train시
생성된 것으로 바뀌어야 함 (달라지지 않았다면 그대로)
- **rewards.py:** source/unitree_rl_lab/unitree_rl_lab/tasks/locomotion/mdp/
isaacclab의 source/isaacclab/isaacclab/envs/mdp/rewards.py 에 없는 추가적 reward 정의
line 120의 foot_clearance_reward는 기존 unitree rl에서 사용됨. 그러나 github에서 학습 속도를 저해한다
는 이슈가 제기되어 line 131에 새로 정의. 다만 현재는 학습에 사용하지 않음 (feet air time으로 대체).
- **curriculums.py:** source/unitree_rl_lab/unitree_rl_lab/tasks/locomotion/mdp/
 - lin_vel_cmd_levels: track_lin_vel_xy의 reward 평균값이 지정된 값보다 크면 linear velocity를 0.1만
큼 증감시킴
 - ang_vel_cmd_levels: 유사함, 사용 X
 - terrain_levels: 로봇이 이동한 거리를 기준으로 terrain level의 증감 결정. velocity_env_cfg의
CurriculumCfg에서 up/down ratio를 지정할 수 있음 (distance vs 특정 거리 * ratio 크기 비교로 결
정)

2. 학습

isaacsim이 설치된 env 실행

unitree_rl_lab dir에서 다음과 같이 학습 코드 실행:

```
python scripts/rsl_rl/train.py --task Unitree-G1-29dof-Velocity --headless --num_envs 4096 --max_iteration 50000 (--video 를 사용해 지정한 interval마다 영상 녹화)
```

학습된 policy의 checkpoint는 unitree_rl_lab/logs/rsl_rl/unitree_g1_29dof_velocity/ 에 실시간으로 저장

다음과 같이 특정 checkpoint 실행:

```
python scripts/rsl_rl/play.py --task Unitree-G1-29dof-Velocity --checkpoint /home/rirolab/unitree_rl_lab/logs/rsl_rl/unitree_g1_29dof_velocity/20XX-XX-XX-xx-xx-xx/model_0000.pt
```

checkpoint 생략 시 가장 최근 checkpoint 실행

Flat terrain의 경우 task 뒤에 Unitree-G1-29dof-Velocity-Flat 을 대신 입력. 현재는 velocity command가 0 으로 설정되었음!

3. Rewards

현재의 reward는 계단을 포함한 rough terrain에 적합함 → 수정 필요!

각 reward에 사용된 함수는 다음 파일에 정의되어 있음

isaacsim/source/isaacsim_tasks/isaacsim_tasks/manager_based/locomotion/velocity/mdp/rewards.py

isaacsim/source/isaacsim/isaacsim/envs/mdp

unitree_rl_lab/source/unitree_rl_lab/unitree_rl_lab/tasks/locomotion/mdp/rewards.py

- **track_lin_vel_xy**: linear velocity command(xy axis)와의 l2 error
로봇을 움직이게 하는 주요 동력이자 학습을 평가하는 요소 중 하나
- **track_ang_vel_z**: angular velocity command(z)와의 l2 error
- **termination_penalty**: early termination에 대한 penalty.
초기에는 weight가 -200이었으나 rough terrain에서 성공률이 낮을 수밖에 없음을 감안해 -150으로 줄임.
rough terrain의 경우 blind 주행 실패 확률이 어느정도(~10%) 있어 penalty가 크게 작용→reward 하락의 주된 원인으로 추정
몇몇 연구에서는 termination penalty 대신 alive reward를 사용하기도 함. 학습 초반에는 termination penalty가 유리하게 작용하는 듯
- **base_linear_velocity**: z방향 linear velocity의 제곱
rough terrain임을 고려하면 유의미한 영향을 주지는 않는 듯함

- **base_angular_velocity**: xy방향 angular velocity의 제공의 합
- **joint_vel**: joint velocity의 l2 sum
- **joint_acc**: joint acceleration의 l2 sum
- **action_rate**: 현재 action과 previous action의 l2 error sum
- **dof_pos_limits**: 현재 joint pos와 soft limit과의 차이(abs)의 합
- **energy**: torque * joint_vel
- **joint_deviation_arms**: default joint angle과의 차이(abs)
- **joint_deviation_waists**
- **joint_deviation_legs**
- **flat_orientation_l2**: projected gravity의 제공의 합
- **base_height**: base의 높이와 target height의 l2 error sum (height scanner를 사용해 terrain 고려)
이 reward를 강하게 주면 무릎을 덜 굽히지 않을까 하는 생각이 듭니다.
- **gait**: 발의 contact sensor input이 설정해둔 phase(period로 설정)와 일치하는지 확인 (논리곱연산)
잘 설정하면 양 발의 비대칭 motion을 교정할 수 있음
- **feet_slide**: 발의 contact 유무 * 발의 linear velocity
- **feet_stumble**: 발에 작용하는 xy방향 force가 z방향 force의 4배보다 큰 경우 penalty
- **feet_air_time**: 한 발로 서있는 시간에 대한 reward (velocity command가 0.1 이하일 땐 0
사람같이 자연스러운 locomotion을 학습하는데 영향을 줌. 다만 학습이 진행될수록 feet air time을 키우기 위
해 한 발로 버티는 시간이 늘어나면서 무릎을 굽히는 현상이 나타남.
- **undesired_contacts**: 발과 발목에 작용하는 threshold 이상의 힘에 대한 penalty
- **alive**: early terminate 되지 않은 경우에 대한 reward
- **feet_clearance**: target height 이상의 발 높이 * foot velocity * weight

		unitree	isaacsim	WMR
1	Tracking Linear Velocity	1.0	1.0	1.0
2	Tracking Angular Velocity	1.0	2.0	1.0
3	Alive	-	-	-
4	Termination	-150.0	-200.0	-200.0
5	Base Linear Velocity(z axis)	-0.5	0	-1.0
6	Energy	-2e-5	-	-0.001
7	Base Angular Velocity	-0.05	-0.05	-0.05
8	Joint Velocity	-0.001	-	-
9	Joint Acceleration	-2.5e-7	-1.25e-7	-2.5e-7
10	Joint Torques	-	-1.5e-7	-
11	Action Rate	-0.01	-0.005	-0.01
12	Base Flat Orientation	-2.0	-1.0	-2.0
13	Joint Position Limit	-2.0	-1.0	-2.0

14	Joint Deviation			
15	Feet AirTime	0.2	0.25	0.2
16	Base Height	-1.0	-	-
17	Gait	1.0	-	-
18	Feet Force	-	-	5e-3
19	Feet Stumble	-1.0	-	-2.0
20	Feet Sliding	-0.2	-0.1	-0.25
21	Flying State	-	-	-1.0
22	Undesired Contacts	-1.0	-	-1.0
23	Feet Clearance	-	-	-

<memo>

BECS 미팅에서 박사분께 들은 내용이기도 하지만, 많이 rough한 지형의 경우 blind로 걷는 것이 비효율적인 것 같습니다. 특히 계단을 포함할 경우 아예 mode switch 되는 식으로 구현을 하거나 camera를 사용하는게 훨씬 효율적이며 하나의 policy로 flat과 rough 모두 안정적으로 걷기 위해서는 reward shaping에 많은 시간을 투자해야 할 것 같아요. 다만 간단한 지형 정도는 조금만 다듬으면 잘 걸을 수 있을 것으로 예상됩니다.

지금 sim2real에서 이슈가 되는 부분인 zero velocity command 상태에서 무릎을 굽히는 동작은, feet air time 이 커짐에 따라 안정성을 확보하기 위해 무릎을 굽히게 되는 것 같습니다. 쉬운 terrain에서 학습이 초반부터 잘 수렴한다면 학습 초반의 overfitting 되지 않은 policy를 사용하는 것이 오히려 나을 수도 있을 것 같습니다.

대략적인 구조

rsl_rl의 onpolicyrunner를 비롯한 코드에서 ppo 학습 진행

→isaacclab의 vecenv_wrapper가 policy에 들어갈 input과 output을 담당함

→이 값들은 isaacclab의 여러 manager를 통해 sim env와 연결됨

→velocity_env_cfg 등의 코드에서 다룰 수 있음

- policy output 및 sim 상의 joint order of actions

```
l hip pitch
r hip pitch
waist yaw
l hip roll
r hip roll
waist roll
l hip yaw
r hip yaw
waist pitch
l knee
r knee
l shoulder pitch
r shoulder pitch
```

l ankle pitch
r ankle pitch
l shoulder roll
r shoulder roll
l ankle roll
r ankle roll

...

4. Sim2Real Deploy

https://github.com/unitreerobotics/unitree_rl_lab

https://github.com/RoboCubPilot/g1_deploy_mujoco : 이걸 누가 만들어놓은 건데 아직 확인 못함
unitree_rl_lab과 isaacsim은 꾸준히 업데이트되므로 최신 버전에서는 아래 방법과 다를 수도 있습니다.

설치

unitree_rl_lab 의 README 따라 설치

unitree_mujoco 설치 시 주의할 점: mujoco version 확인, 설치 후 simulate/config.yaml 수정

unitree_rl_lab 과 isaacsim 연결 확인: python scripts/list_envs.py → 학습용, 순수 deploy 단계에서는 불필요

deploy steps

1. deploy/robots/g1_29dof/config/policy/velocity/v0/exported/ 에 사용할 policy.onnx 넣어놓기
g1_29dof/config/config.yaml 마지막 줄 policy_dir 확인

```
Velocity:  
policy_dir: config/policy/velocity  
# policy_dir: ../../logs/rsl_rl/unitree_g1_29dof_velocity
```

2. deploy/robots/g1_29dof/config/policy/velocity/v0/params/deploy.yaml 의 observation 항목 중
velocity_command 대신 keyboard_velocity_commands 사용 (joystick 사용시에는 다시 원래대로)

```
# velocity_commands:  
keyboard_velocity_commands:  
  params: {command_name: base_velocity}  
  clip: null  
  scale: [1.0, 1.0, 1.0]  
  ...
```

이 observation은 deploy/robots/g1_29dof/src/State_RLBase.cpp에서 설정됨

```

REGISTER_OBSERVATION(keyboard_velocity_commands)
{
    std::string key = FSMState::keyboard→key();
    static auto cfg = env→cfg["commands"]["base_velocity"]["ranges"];

    static std::unordered_map<std::string, std::vector<float>> key_commands = {
        {"w", {1.0f, 0.0f, 0.0f}},
        {"s", {-1.0f, 0.0f, 0.0f}},
        {"a", {0.0f, 1.0f, 0.0f}},
        {"d", {0.0f, -1.0f, 0.0f}},
        {"q", {0.0f, 0.0f, 1.0f}},
        {"e", {0.0f, 0.0f, -1.0f}}
    };
    std::vector<float> cmd = {0.0f, 0.0f, 0.0f};
    if (key_commands.find(key) != key_commands.end())
    {
        // TODO: smooth and limit the velocity commands
        cmd = key_commands[key];
    }
    return cmd;
}

```

yaml의 scale을 건드리면 속도를 조절할 수 있을 것 같네요

3. deploy/robots/g1_29dof/main.cpp

```

// Initialize FSM
auto & joy = FSMState::lowstate→joystick;
auto fsm = std::make_unique<CtrlFSM>(new State_Passive(FSMMode::Passive));
fsm→states.back()→registered_checks.emplace_back(
    std::make_pair(
        [&]()→bool{ return joy.LT.pressed && joy.up.on_pressed; }, // L2 + Up
        (int)FSMMode::FixStand
    )
);
fsm→add(new State_FixStand(FSMMode::FixStand));
fsm→states.back()→registered_checks.emplace_back(
    std::make_pair(
        [&]()→bool{ return joy.RB.pressed && joy.X.on_pressed; }, // R1 + X
        FSMMode::Velocity
    )
);
fsm→add(new State_RLBase(FSMMode::Velocity, "Velocity"));

std::cout << "Press [L2 + Up] to enter FixStand mode.\n";
std::cout << "And then press [R1 + X] to start controlling the robot.\n";

```

```
while (true)
{
    sleep(1);
}
```

현재(2025.10.13) keyboard로 모드 변경할 수 있도록 바꿔놓음

joystick으로 모드 변경하려면 위의 예시처럼 다시 바꿔야 함

4. deploy/include/FSM/State_Passive.h 수정 (완료)

```
for(int i(0); i < kd.size(); ++i)
{
    auto & motor = lowcmd->msg_.motor_cmd()[i];
    motor.mode() = 1; # ← Add this line
    motor.kp() = 0;
    motor.kd() = kd[i];
    motor.dq() = 0;
    motor.tau() = 0;
}
```

5. cmake

```
cd unitree_rl_lab/deploy/robots/g1_29dof # or other robots
mkdir build && cd build
cmake .. && make
```

6. Mujoco Sim2Sim (non necessary)

다른 터미널에서 아래 코드 실행 → mujoco 창 open

! simulate/config.yaml 수정 필수 !

```
# start simulation
cd unitree_mujoco/simulate/build
./unitree_mujoco
# ./unitree_mujoco -i 0 -n eth0 -r g1 -s scene_29dof.xml # alternative
```

unitree_rl_lab 터미널에서 아래 코드 실행

```
cd unitree_rl_lab/deploy/robots/g1_29dof/build
./g1_ctrl --network lo
# 1. press [L2 + Up] to set the robot to stand up
# 2. Click the mujoco window, and then press 8 to make the robot feet touch the ground.
# 3. Press [R1 + X] to run the policy.
# 4. Click the mujoco window, and then press 9 to disable the elastic band.
```

7. G1 부팅

Quick Manual: https://support.unitree.com/home/en/G1_developer/quick_start

팔을 제자리에 돌려놓고 전원 on: 짧게 한 번 길게 한 번
리모컨 on: 짧게 한 번 길게 한 번
잠시 대기 후 L1+A 눌러 damping mode 진입
L2+R2 눌러 debug mode(develop mode) 진입 → 헤드에 녹색등
L2+A 누르면 기본 자세, L2+B 누르면 damping state.
종료 시 damping state에서 버튼 눌러 종료

8. G1 연결

Manual: https://support.unitree.com/home/en/G1_developer/quick_development

Ethernet 연결: 현재 워크에서는 eno1에 g1_locomotion 연결하면 됨

ping 192.168.123.161로 test

deploy: 긴급 종료시 Ctrl+C

```
cd unitree_rl_lab/deploy/robots/g1_29dof/build
./g1_ctrl --network eno1
# 1. press [L2 + Up] to set the robot to stand up
# 2. Click the mujoco window, and then press 8 to make the robot feet touch the ground.
# 3. Press [R1 + X] to run the policy.
# 4. Click the mujoco window, and then press 9 to disable the elastic band.
```

TMI: terrain 변경 방법

source/unitree_rl_lab/unitree_rl_lab/tasks/locomotion/robots/g1/29dof/velocity_env_cfg.py에 추가

```
from isaacsim.terrains.config.rough import ROUGH_TERRAINS_CFG # isort: skip
```

```
class RobotSceneCfg(InteractiveSceneCfg):
    """Configuration for the terrain scene with a legged robot."""

    # ground terrain
    terrain = TerrainImporterCfg(
        prim_path="/World/ground",
        terrain_type="generator", # "plane", "generator"
        # terrain_generator=COBBLESTONE_ROAD_CFG, # None, ROUGH_TERRAINS_CFG
        # max_init_terrain_level=COBBLESTONE_ROAD_CFG.num_rows - 1,

        terrain_generator=ROUGH_TERRAINS_CFG,
        max_init_terrain_level=5,

        collision_group=-1,
        physics_material=sim_utils.RigidBodyMaterialCfg(
            friction_combine_mode="multiply",
            restitution_combine_mode="multiply",
```

```
static_friction=1.0,  
dynamic_friction=1.0,  
)
```